# Job Board Platform

## Technical Documentation

Full-Stack Web Application

FastAPI | React | PostgreSQL | Docker

*Version 1.0 - February 2026*

*https://github.com/bekgm/job_finding_service*

# 1. Project Overview

The Job Board Platform is a production-ready full-stack web application that connects employers with job candidates. Employers can create company profiles, post job listings, and manage incoming applications. Candidates can browse jobs with advanced filtering, apply with cover letters and PDF resumes, and track their application status in real time.

The application follows modern best practices including async I/O throughout the backend, JWT-based authentication with refresh tokens, role-based access control, and a responsive single-page frontend.

## Key Features

- JWT authentication with access/refresh token pair and silent refresh
- Role-based access control (Candidate and Employer roles)
- Company profile management (one company per employer)
- Job listings with full-text search, filters, and server-side pagination
- Job application with cover letter and PDF resume file upload
- Application status workflow: pending, reviewed, shortlisted, rejected, accepted
- Responsive UI with TailwindCSS
- Dockerized deployment with Docker Compose (3 services)

# 2. Technology Stack

## Backend

| Technology | Version | Purpose |
|---|---|---|
| FastAPI | 0.115.x | Async web framework |
| Python | 3.12 | Programming language |
| PostgreSQL | 16 | Relational database |
| SQLAlchemy | 2.0 (async) | ORM with asyncpg driver |
| Alembic | 1.14.x | Database migrations |
| Pydantic | v2 | Data validation & serialization |
| python-jose | 3.3.x | JWT token handling |
| bcrypt | 4.x | Password hashing |

## Frontend

| Technology | Version | Purpose |
|---|---|---|
| React | 18.x | UI component library |
| TypeScript | 5.x | Type-safe JavaScript |
| Vite | 5.x | Build tool & dev server |
| TailwindCSS | v4 | Utility-first CSS framework |
| React Router | v6 | Client-side routing |
| TanStack Query | v5 | Server state management |
| Zustand | 4.x | Client state management |
| Axios | 1.x | HTTP client with interceptors |

## Infrastructure

| Technology | Version | Purpose |
|---|---|---|
| Docker | latest | Containerization |
| Docker Compose | v2 | Multi-service orchestration |
| Nginx | alpine | Frontend server & reverse proxy |

# 3. System Architecture

The system is composed of three Docker containers orchestrated via Docker Compose:

| Service | Port | Description |
|---|---|---|
| Frontend | 3000 -> 80 | React SPA served by Nginx |
| Backend API | 8000 | FastAPI async application |
| Database | 5433 -> 5432 | PostgreSQL 16 Alpine |

## Backend Architecture Pattern

The backend follows a clean layered architecture pattern:

Router -> Service -> Repository -> Model

- Routers handle HTTP request/response mapping and dependency injection.
- Services contain business logic, validation, and orchestration.
- Repositories encapsulate database queries using async SQLAlchemy.
- Models define the ORM schema and relationships.

## Frontend Architecture Pattern

The frontend follows a structured component architecture:

Pages -> API Layer -> Axios Instance -> Backend

- Pages are route-level components that compose the UI.
- API Layer provides typed functions for each endpoint.
- Axios Instance manages JWT tokens via request/response interceptors.
- Zustand Store maintains authentication state globally.
- React Query handles server state caching, refetching, and mutations.

## Authentication Flow

1. User registers or logs in -> server returns access_token + refresh_token.
2. Axios interceptor attaches Bearer access_token to every request.
3. On 401 response, interceptor automatically attempts silent refresh.
4. If refresh succeeds, the failed request is retried transparently.
5. If refresh fails, user is logged out and redirected to login page.
6. Concurrent requests during refresh are queued to prevent race conditions.

# 4. Database Schema

## Users Table

| Column | Type | Null | Description |
|---|---|---|---|
| id | UUID (PK) | No | Auto-generated primary key |
| email | VARCHAR(320) | No | Unique, indexed email address |
| hashed_password | VARCHAR(1024) | No | bcrypt password hash |
| full_name | VARCHAR(255) | No | User display name |
| role | ENUM | No | candidate or employer |
| is_active | BOOLEAN | No | Account active flag (default true) |
| created_at | TIMESTAMP | No | Auto-set on creation |
| updated_at | TIMESTAMP | No | Auto-updated on modification |

## Companies Table

| Column | Type | Null | Description |
|---|---|---|---|
| id | UUID (PK) | No | Auto-generated primary key |
| name | VARCHAR(255) | No | Company name |
| description | TEXT | Yes | Company description |
| website | VARCHAR(512) | Yes | Company website URL |
| location | VARCHAR(255) | Yes | Company location |
| owner_id | UUID (FK) | No | Unique ref to users.id (1:1) |
| created_at | TIMESTAMP | No | Auto-set on creation |
| updated_at | TIMESTAMP | No | Auto-updated on modification |

## Jobs Table

| Column | Type | Null | Description |
|---|---|---|---|
| id | UUID (PK) | No | Auto-generated primary key |
| title | VARCHAR(255) | No | Job title |
| description | TEXT | No | Full job description |
| location | VARCHAR(255) | Yes | Job location |
| is_remote | BOOLEAN | No | Remote work flag |
| job_type | ENUM | No | full_time/part_time/contract/intern |
| salary_min | INTEGER | Yes | Minimum salary |
| salary_max | INTEGER | Yes | Maximum salary |
| is_active | BOOLEAN | No | Listing active flag |
| company_id | UUID (FK) | No | Reference to companies.id |

| | | | |
|---|---|---|---|
| created_at | TIMESTAMP | No | Auto-set on creation |
| updated_at | TIMESTAMP | No | Auto-updated on modification |

## Applications Table

| Column | Type | Null | Description |
|---|---|---|---|
| id | UUID (PK) | No | Auto-generated primary key |
| candidate_id | UUID (FK) | No | Reference to users.id |
| job_id | UUID (FK) | No | Reference to jobs.id |
| status | ENUM | No | pending/reviewed/shortlisted/... |
| cover_letter | TEXT | Yes | Candidate cover letter |
| resume_path | VARCHAR(512) | Yes | Path to uploaded resume |
| created_at | TIMESTAMP | No | Auto-set on creation |
| updated_at | TIMESTAMP | No | Auto-updated on modification |

## Enum Definitions

| Enum | Values | Used In |
|---|---|---|
| UserRole | candidate, employer | users.role |
| JobType | full_time, part_time, ... | jobs.job_type |
| ApplicationStatus | pending, reviewed, ... | applications.status |

# 5. API Reference

All endpoints are prefixed with /api. Authentication uses Bearer JWT tokens in the Authorization header. The full OpenAPI 3.0 specification is available in openapi.yml for import into Postman or Swagger UI.

## Auth Endpoints (/api/auth)

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | /api/auth/register | Register new user | No |
| POST | /api/auth/login | Login, get JWT tokens | No |
| POST | /api/auth/refresh | Refresh access token | No |
| GET | /api/auth/me | Get current user | Yes (any) |

## Company Endpoints (/api/companies)

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | /api/companies | Create company | Employer |
| GET | /api/companies/me | Get my company | Yes (any) |
| PATCH | /api/companies/me | Update my company | Employer |
| GET | /api/companies/{id} | Get company by ID | No |

## Job Endpoints (/api/jobs)

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| GET | /api/jobs | List/filter/search jobs | No |
| POST | /api/jobs | Create job listing | Employer |
| GET | /api/jobs/employer/my-jobs | My posted jobs | Employer |
| GET | /api/jobs/{id} | Get job details | No |
| PATCH | /api/jobs/{id} | Update job | Employer (owner) |
| DELETE | /api/jobs/{id} | Delete job | Employer (owner) |

## Application Endpoints (/api/applications)

| Method | Endpoint | Description | Auth Required |
|---|---|---|---|
| POST | /apps/jobs/{id}/apply | Apply to job | Candidate |
| GET | /api/applications/me | My applications | Candidate |
| GET | /apps/jobs/{id} | Job's applications | Employer |
| PATCH | /apps/{id}/status | Update app status | Employer |

## 6. Job Search & Filtering

The GET /api/jobs endpoint supports the following query parameters:

| Parameter | Type | Default | Description |
|---|---|---|---|
| page | integer | 1 | Page number (1-indexed) |
| size | integer | 20 | Items per page (1-100) |
| search | string | - | Search in title and description |
| is_remote | boolean | - | Filter by remote status |
| job_type | enum | - | Filter by job type |
| salary_min | integer | - | Minimum salary filter |
| salary_max | integer | - | Maximum salary filter |

Example request:

```
GET /api/jobs?search=python&is_remote=true&job_type=full_time&page=1&size=10
```

Example response:

```
{
  "items": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "title": "Senior Python Developer",
      "description": "...",
      "is_remote": true,
      "job_type": "full_time",
      "salary_min": 3000,
      "salary_max": 6000,
      "company": { "name": "Acme Corp", ... }
    }
  ],
  "total": 42,
  "page": 1,
  "size": 10,
  "pages": 5
}
```

# 7. Deployment Guide

## Prerequisites

- Docker Desktop installed and running
- Git installed
- Ports 3000, 5433, and 8000 available

## Step 1: Clone the Repository

```
git clone https://github.com/bekgm/job_finding_service.git
cd job_finding_service
```

## Step 2: Configure Environment (Optional)

```
cp backend/.env.example backend/.env
```

Default values work for local development without changes.

## Step 3: Build and Start Services

```
docker-compose up --build -d
```

This builds and starts 3 containers: PostgreSQL database, FastAPI backend, and Nginx-served React frontend.

## Step 4: Run Database Migrations

```
docker-compose exec api alembic upgrade head
```

## Step 5: Access the Application

| Service | URL | Description |
|---|---|---|
| Frontend | http://localhost:3000 | React web application |
| Backend API | http://localhost:8000 | FastAPI with Swagger |
| API Docs | http://localhost:8000/docs | Interactive API docs |

## Stopping and Cleanup

```
# Stop services
docker-compose down

# Stop and remove all data
docker-compose down -v
```

## 8. Environment Variables

Configured via backend/.env file or Docker environment:

| Variable | Default | Description |
|---|---|---|
| DATABASE_URL | postgresql+asyncpg://... | Database connection string |
| SECRET_KEY | change-me-in-production | JWT signing secret key |
| ALGORITHM | HS256 | JWT algorithm |
| ACCESS_TOKEN_EXPIRE_MIN | 30 | Access token TTL (minutes) |
| REFRESH_TOKEN_EXPIRE_DAYS | 7 | Refresh token TTL (days) |
| UPLOAD_DIR | uploads | Resume upload directory |
| MAX_UPLOAD_SIZE_MB | 5 | Max file upload size (MB) |
| POSTGRES_USER | postgres | Database username |
| POSTGRES_PASSWORD | postgres | Database password |
| POSTGRES_DB | jobboard | Database name |

## 9. Project File Structure

```
job_finding_service/
+-- backend/
|   +-- app/
|   |   +-- core/          # Config, security, exceptions, deps
|   |   +-- db/            # Database engine & base model
|   |   +-- models/        # SQLAlchemy ORM models (4 models)
|   |   +-- schemas/       # Pydantic v2 schemas
|   |   +-- repositories/  # Data access layer
|   |   +-- services/      # Business logic layer
|   |   +-- routers/       # FastAPI route handlers
|   |   +-- utils/         # Pagination helper
|   |   +-- main.py        # App entrypoint
|   +-- alembic/           # DB migrations
|   +-- Dockerfile
|   +-- requirements.txt
|   +-- .env.example
+-- frontend/
|   +-- src/
|   |   +-- api/           # API service functions
|   |   +-- components/    # Shared UI (6 components)
|   |   +-- pages/         # Route pages (8 pages)
|   |   +-- stores/        # Zustand auth store
|   |   +-- lib/           # Axios with JWT interceptors
|   |   +-- types/         # TypeScript definitions
|   |   +-- App.tsx        # Route config
|   |   +-- main.tsx       # React entry
|   +-- Dockerfile         # Multi-stage (Node -> Nginx)
|   +-- nginx.conf
|   +-- package.json
+-- docker-compose.yml
+-- openapi.yml            # OpenAPI 3.0 spec
+-- README.md
```

# 10. Testing with Postman

## Importing the API

1. Open Postman and click Import.
2. Upload the openapi.yml file from the project root.
3. Select 'Postman Collection' as the output format.
4. Click Import - all 15 endpoints will be organized by tag.

## Testing Workflow

### Step 1: Register an Employer

```
POST /api/auth/register
{
  "email": "employer@example.com",
  "password": "Test1234!",
  "full_name": "Bob Employer",
  "role": "employer"
}
```

### Step 2: Login

```
POST /api/auth/login
{
  "email": "employer@example.com",
  "password": "Test1234!"
}
```

Copy the access_token from the response and set it as Bearer token.

### Step 3: Create a Company

```
POST /api/companies
Authorization: Bearer <access_token>
{
  "name": "Acme Corp",
  "description": "Tech company",
  "location": "Almaty, KZ"
}
```

### Step 4: Post a Job

```
POST /api/jobs
Authorization: Bearer <access_token>
{
  "title": "Backend Developer",
  "description": "Python/FastAPI developer needed",
  "is_remote": true,
  "job_type": "full_time",
  "salary_min": 3000,
  "salary_max": 6000
}
```

### Step 5: Register a Candidate & Apply

Register a new user with role 'candidate', login to get their token, then use POST /api/applications/jobs/{job_id}/apply with a multipart form containing cover_letter and an optional resume PDF file.